

1. Introdução

Contexto do projeto

Este projeto, parte da Unidade 2, visou criar uma calculadora avançada em C. O objetivo foi aplicar conceitos como manipulação de strings, repetições aninhadas, matrizes, ponteiros e alocação dinâmica de memória para aprimorar a calculadora desenvolvida na Unidade 1.

Objetivos da U2

O objetivo para esta unidade foi a aplicação prática dos conceitos de manipulação de strings, estruturas de repetição aninhadas, matrizes, ponteiros e alocação dinâmica de memória.

Funcionalidades implementadas (em negrito)

- Calculadora básica
Além das 4 operações, inclui potenciação e prioridade com parêntesis.
- Calculadora científica
 - Funções trigonométricas: $\sin$, $\cos$ e $\tan$.
 - Logaritmos: $\log_{10}$ ou $\ln$.
 - Raiz quadrada
 - Potenciação
 - Fatorial
- Operações com Matrizes
 - Soma
 - Multiplicação
 - Transposição
- Operações com Strings

- Comparar (`strcmp`)
- Copiar (`strcpy`)
- Concatenar (`strcat`)
- Obter tamanho (`strlen`)
- Inverter string
- Contar vogais/consoantes

- Mostrar histórico

- Limpar histórico

2. Metodologia

Ferramentas utilizadas

O projeto foi desenvolvido em Windows 10, compilado com TDM-GCC 9.2.0 (64-bit) e editado na IDE Embarcadero Dev-C++ 6.3.

Abordagem de desenvolvimento

Adotou-se uma abordagem modular e procedural, separando as responsabilidades do programa em funções distintas e bem definidas, como `calculadoraMatricial()`, `adicionarAoHistorico()` e `avaliarExpressao()`, o que tornou o código mais legível e de fácil depuração.

Organização do código

A organização do código seguiu uma estrutura lógica: primeiro, as definições de constantes e estruturas; em seguida, os protótipos das funções, a função `main` e, por fim, a implementação detalhada de cada função, agrupadas por funcionalidade.

3. Análise do Código

Strings

Como foram utilizadas no projeto?

Strings foram usadas para interagir com o usuário, capturando expressões na calculadora básica (`scanf("%[^\\n]", expressao)`) e nomes de funções na científica. Também foram essenciais para armazenar operações no histórico dinâmico, criando uma interface de texto flexível.

Quais funções de manipulação foram implementadas?

Foram implementadas funções da biblioteca `<string.h>`, principalmente na funcionalidade de manipulação de strings. Utilizou-se `strcpy` para copiar expressões para o histórico, `strcmp` para comparar entradas do usuário, `strcat` para concatenar e `strlen` para obter o tamanho de strings em validações.

Exemplos de uso

Um exemplo prático está na função `adicionarAoHistorico`, onde `strcpy(historico.itens[historico.contador].expressao, expressao)` copia a expressão para o histórico. Outro exemplo é a inversão de string, onde ponteiros foram usados para trocar os caracteres das extremidades até o centro.

Estruturas de Repetição Aninhadas

Onde foram aplicadas?

As repetições aninhadas foram cruciais no módulo de "Operações com Matrizes" para percorrer as estruturas bidimensionais, acessando cada elemento por seu índice. Funções como `preencherMatriz`, `imprimirMatriz` e

`somarMatrizes` usam dois laços `for` aninhados.

Qual a complexidade dos algoritmos?

A complexidade dos algoritmos variou. Soma e transposição foram mais simples, exigindo apenas mapeamento de índices. A multiplicação foi o maior desafio, com três laços aninhados e a restrição de que o número de colunas da primeira matriz fosse igual ao de linhas da segunda, exigindo cuidado com a lógica $A[i][k] \times B[k][j]$.

Casos de uso

Na função `multiplicarMatrizes`, o laço externo itera pelas linhas de A, o intermediário pelas colunas de B, e o interno realiza o somatório dos produtos dos elementos correspondentes para calcular o resultado.

Matrizes

Como foram implementadas?

As matrizes foram implementadas com alocação dinâmica e ponteiros duplos (`double**`). A estrutura `Matriz` armazena um ponteiro para um array de ponteiros, onde cada um aponta para uma linha. A função `criarMatriz` gerencia essa alocação em duas etapas.

Quais operações são suportadas?

O projeto suporta soma, multiplicação e transposição de matrizes. Cada operação possui uma função específica que recebe as matrizes de entrada (como ponteiros) e retorna uma nova matriz com o resultado, alocada dinamicamente.

Estratégias de percorrimento

A estratégia de percorrimento utilizou laços `for` aninhados. O laço externo (variável `i`) itera sobre as linhas, e o interno (variável `j`) sobre as colunas. O acesso a um elemento é feito via `m->dados[i][j]`.

Ponteiros

Como foram utilizados?

Ponteiros foram centrais para gerenciar memória dinâmica e otimizar a passagem de dados, sendo a base das estruturas `Matriz` (`double** dados`) e `HistoricoDinamico` (`HistoricoItem* itens`). Ponteiros de `char` também foram usados para manipular strings eficientemente.

Passagem por referência vs. valor

A diferença é clara no código: estruturas grandes como `Matriz` são passadas por referência (`Matriz* m`) para evitar cópias desnecessárias e aumentar a eficiência. Tipos simples como `int` ou `double` são passados por valor.

Exemplos práticos

A função `liberarMatriz(Matriz* m)` recebe um ponteiro para liberar a memória alocada originalmente. Na função `mostrarHistorico`, a aritmética de ponteiros (`ptr++`) foi usada para percorrer o array de histórico sem a necessidade de índices.

Alocação Dinâmica

Onde foi necessária?

Foi necessária no gerenciamento do histórico de cálculos, que precisava crescer dinamicamente, e na criação de matrizes, cujas dimen-

sões são definidas pelo usuário em tempo de execução.

Como é gerenciada a memória?

A memória foi gerenciada com funções da `<stdlib.h>`: `malloc` para alocação inicial, `realloc` para expandir o histórico quando necessário, e `free` para liberar a memória não utilizada.

Tratamento de falhas de alocação

Após cada chamada a `malloc` ou `realloc`, o ponteiro retornado foi verificado. Se fosse `NULL`, uma mensagem de erro era exibida e, em casos críticos, o programa era encerrado. Rotinas de limpeza foram implementadas para falhas parciais.

Estratégia para evitar *memory leaks*

A estratégia foi garantir que toda memória alocada com `malloc` ou `realloc` fosse liberada com `free`. Funções como `liberarHistorico` e `liberarMatriz` foram criadas para garantir que toda a memória associada a uma estrutura fosse devolvida ao sistema.

4. Dificuldades e Soluções

Os principais desafios foram o gerenciamento manual de memória com ponteiros, especialmente em matrizes dinâmicas, e evitar vazamentos de memória. A implementação do algoritmo *Shunting-Yard* para avaliar expressões matemáticas também foi complexa, exigindo bom entendimento de pilhas e precedência de operadores. Para superar isso, adotei uma abordagem de testes e depuração, validando cada função isoladamente.

5. Conclusão

Este projeto consolidou os conceitos da Unidade 2, aplicando-os em um sistema prático. A implementação da calculadora demonstrou como strings, matrizes, ponteiros e alocação dinâmica trabalham em conjunto para criar um software eficiente. As lições aprendidas sobre gerenciamento de memória serão fundamentais para projetos futuros mais complexos.

Perguntas Orientadoras

1. Quais conceitos da U2 foram aplicados no projeto?

Foram aplicados todos os conceitos centrais da U2: manipulação de strings, estruturas de repetição aninhadas para matrizes, implementação de matrizes dinâmicas com ponteiros duplos, uso de ponteiros para gerenciamento de memória e passagem por referência, e alocação dinâmica (`malloc`, `realloc`, `free`).

2. Como a organização do código facilita a manutenção?

O código foi organizado de forma modular, com funções específicas para cada tarefa. O uso de protótipos e o agrupamento por funcionalidade tornam o código legível e fácil de navegar, permitindo que uma parte do sistema seja modificada sem impactar as demais.

3. Quais foram os principais desafios técnicos enfrentados?

Os maiores desafios foram o gerenciamento de memória em matrizes dinâmicas, garantindo a liberação correta para evitar vazamentos, e a implementação do algoritmo *Shunting-*

Yard, que exigiu um bom entendimento de pilhas e precedência de operadores.

4. Como foram implementadas as estruturas de dados complexas (matrizes)?

As matrizes foram implementadas usando uma estrutura (`Matriz`) com as dimensões e um ponteiro duplo (`double** dados`). A alocação ocorre em duas etapas: primeiro para as linhas (array de ponteiros) e depois para as colunas de cada linha (array de `double`).

5. Qual a estratégia para gerenciamento de memória?

A estratégia foi o gerenciamento manual e cuidadoso, usando `malloc` para alocação, `realloc` para expansão e `free` para liberação. Funções específicas de criação e liberação (ex: `criarMatriz`, `liberarMatriz`) encapsularam essa lógica.

6. Como você garante que não há vazamentos de memória?

Garante-se a ausência de vazamentos ao corresponder cada `malloc` ou `realloc` com uma chamada `free`. Funções de liberação, como `liberarMatriz`, desalocam a memória de forma inversa à alocação para garantir que nada seja perdido.

7. Quais vantagens a alocação dinâmica trouxe para seu projeto?

A alocação dinâmica trouxe flexibilidade, permitindo criar matrizes de qualquer dimensão em tempo de execução e um histórico de cálculos que cresce conforme a necessidade, evitando o desperdício de memória de arrays estáticos.

8. Como os ponteiros foram utilizados para melhorar a eficiência?

Ponteiros evitaram a cópia de grandes estruturas de dados. Ao passar um ponteiro de uma Matriz (Matriz*), apenas o endereço é copiado, economizando memória e tempo de processamento. A aritmética de ponteiros também permitiu percorrer arrays de forma mais direta.